

FARMEASE APPLICATION

Project Report

NAME : ABHISHEK NACHANEKAR

STUDENT ID : 05250140

COURSE ID : ANP-D4854

GITHUB REPOSITORY : <https://github.com/abhisheknachanekar-dev/Sprint1>

INDEX

Sr. No.	Title	Page No.
1	Introduction	3
2	Abstract	4
3	Technologies Used	5
4	Database	6
5	ER Diagram	7
6	Backend	9
7	Frontend	20
8	Application Features	24
9	Future Scope	25
10	Acknowledgement	26
11	Bibliography	27

INTRODUCTION

Agriculture is an important part of the economy and supports a large number of farmers, buyers, service providers, and other participants in the agricultural supply chain. Farmers may face difficulties in reaching customers, finding agricultural services, managing orders, and maintaining information about products and bookings through a single platform. FarmEase is a web-based application designed to provide a centralized digital platform for agricultural product purchasing and agriculture-related services.

The FarmEase application provides a marketplace where users can view agricultural products such as Tomatoes, Wheat, Potatoes, Bitter Gourd, and Flowers. Users can select a product, view its details, specify quantity, provide delivery information, select a payment method, and place an order. The application also includes service-related functionality through which users can view available services and create service bookings.

The system includes user registration and login functionality. User registration is connected to a MySQL database through a Node.js and Express.js backend. Passwords are protected using bcrypt hashing, and the frontend communicates with the backend through JSON APIs. The backend is separated from the frontend and provides the database operations required by the application.

The project is organized into separate backend and frontend folders. The frontend uses HTML, CSS, and JavaScript to create the user interface, while Node.js, Express.js, mysql2, and MySQL are used for server-side processing and data management. The main purpose of FarmEase is to demonstrate how web technologies and database systems can be combined to create a practical agriculture marketplace and service-management platform.

ABSTRACT

FarmEase is a web-based agriculture marketplace and services application developed to connect agricultural products and services with users through a centralized online platform. The system provides features for user registration, login, product browsing, product purchasing, order management, service browsing, and service booking.

The marketplace allows users to browse products and select an individual product for purchase. The product details page displays information such as product name, image, description, price, quantity, delivery charge, and total amount. Users can enter delivery details and select an available payment method such as UPI, card, or cash on delivery before placing an order.

The application uses a Node.js and Express.js backend with MySQL as the relational database. The database contains tables for users, products, orders, order_items, services, and service_bookings. Passwords are stored using bcrypt hashing rather than plain text. The frontend communicates with backend APIs using JavaScript fetch requests.

FarmEase demonstrates the integration of frontend development, backend API development, authentication, relational database management, order processing, and service booking in a single web application. The system can be extended in the future with features such as online payment gateway integration, order tracking, farmer/vendor dashboards, notifications, reviews, multilingual support, and mobile application support.

TECHNOLOGIES USED

3.1 HTML: HTML (HyperText Markup Language) is used to create the basic structure of the FarmEase web pages. It is used for navigation, forms, product sections, service sections, buttons, order forms, registration and login interfaces, and other webpage elements.

3.2 CSS: CSS (Cascading Style Sheets) is used to control the visual appearance and layout of the application. It is used for colors, typography, spacing, product cards, forms, buttons, navigation areas, responsive layouts, and the overall visual design.

3.3 JavaScript: JavaScript provides client-side functionality and interactivity. It is used for form handling, product selection, reading URL parameters, calculating quantities and totals, communicating with backend APIs, and dynamically displaying data.

3.4 Node.js: Node.js is the server-side JavaScript runtime used to execute the FarmEase backend. It provides the environment for processing requests, implementing application logic, and communicating with MySQL.

3.5 Express.js: Express.js is the backend web framework used with Node.js. It is used to create API routes for registration, authentication, products, orders, and other application operations.

3.6 MySQL: MySQL is the relational database management system used by FarmEase to store structured application data. It stores user accounts, products, orders, order items, services, and service bookings.

3.7 mysql2: The mysql2 Node.js package is used to connect the Express backend with the MySQL database and execute SQL queries.

3.8 bcrypt: bcrypt is used for password hashing. During registration, the user's password is converted into a secure hash before it is stored in the database. During login, bcrypt.compare is used to verify the entered password against the stored hash.

3.9 CORS: CORS (Cross-Origin Resource Sharing) is used by the backend to allow communication between the separate frontend and backend applications when they are running on different origins or ports.

DATABASE

FarmEase uses a MySQL database named 'farmease' for persistent storage. The database is designed to keep user information, product information, orders, order items, agricultural services, and service bookings in structured relational tables.

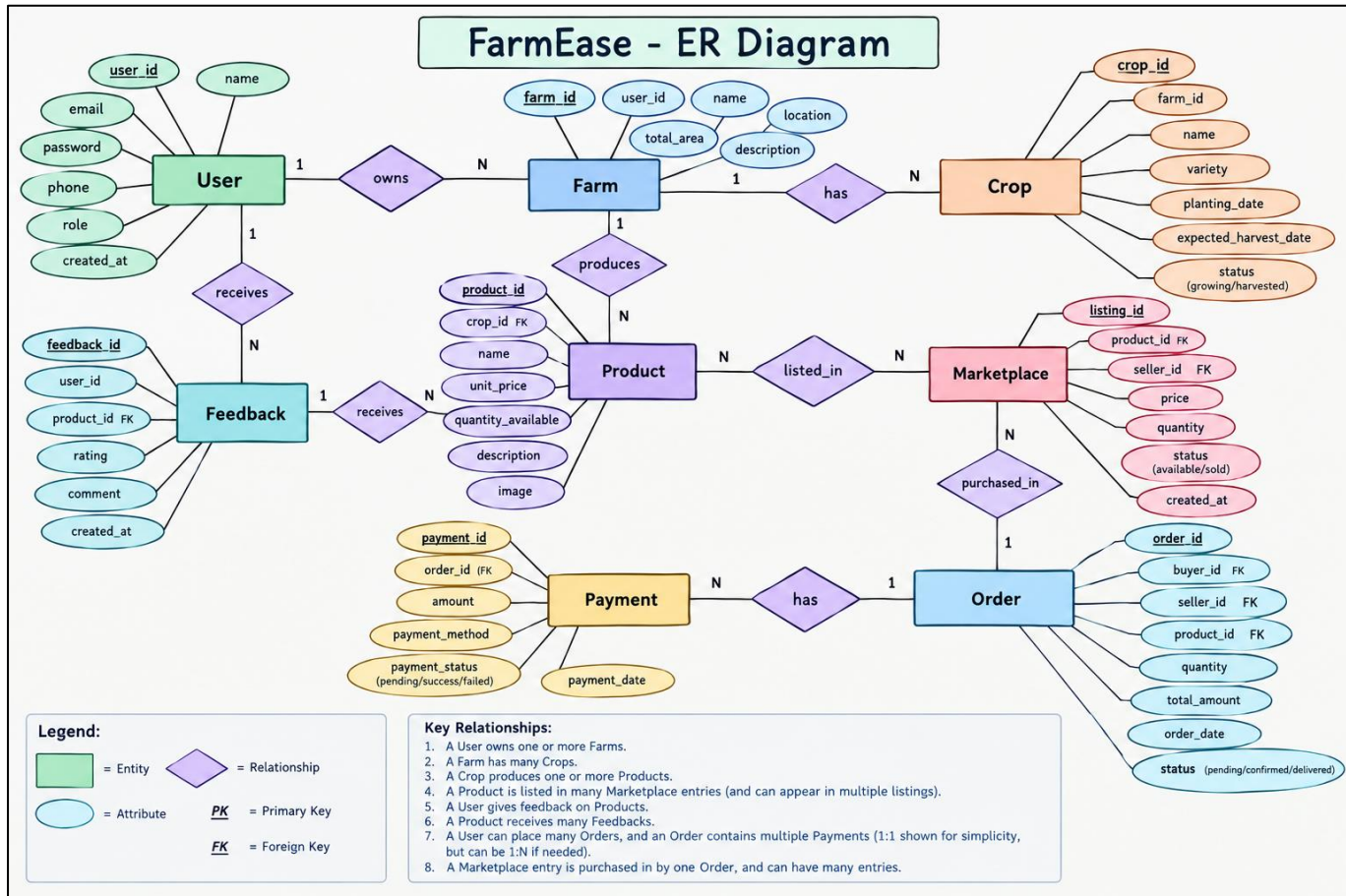
The main database tables are:

Table	Purpose	Important Data
users	Stores registered user accounts.	User ID, name, email, password hash
products	Stores agricultural marketplace products.	Product ID, name, description, price, product details
orders	Stores customer orders.	Order ID, user ID, status, payment status, delivery address
order_items	Stores individual products included in an order.	Order item ID, order ID, product ID, quantity, price
services	Stores agriculture-related services.	Service ID, service name, description, price/details
service_bookings	Stores bookings made for services.	Booking ID, user ID, service ID, booking information, status

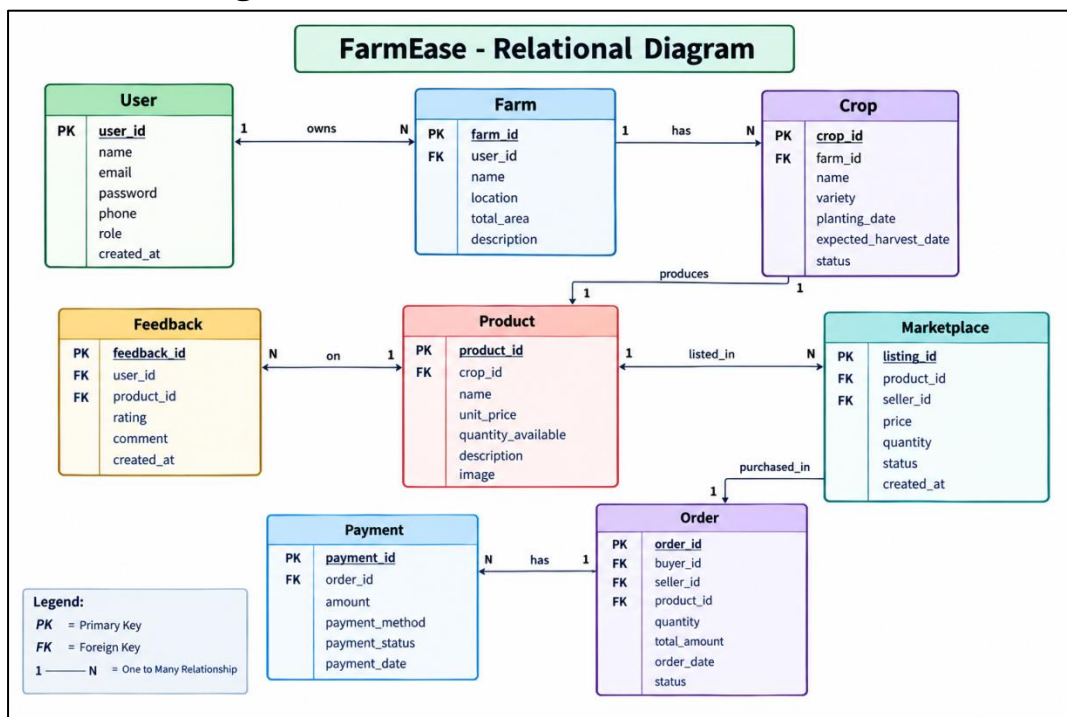
The database design separates users, products, orders, and service information so that the same data can be reused without unnecessary duplication. Orders and order_items are separated so that one order can contain multiple products.

```
mysql> use farmease;
Database changed
mysql> show tables;
+-----+
| Tables_in_farmease |
+-----+
| order_items         |
| orders              |
| products            |
| service_bookings    |
| services            |
| users               |
+-----+
6 rows in set (0.03 sec)
```

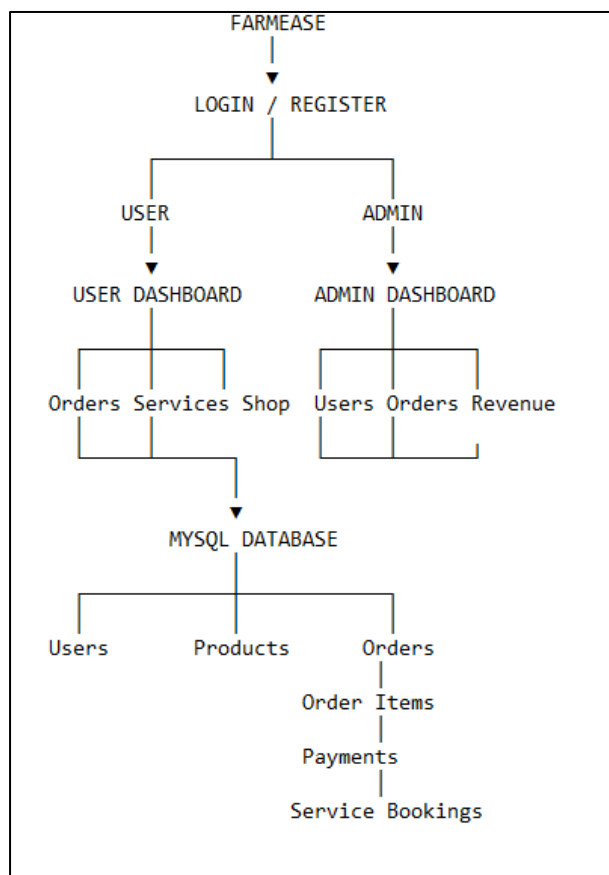
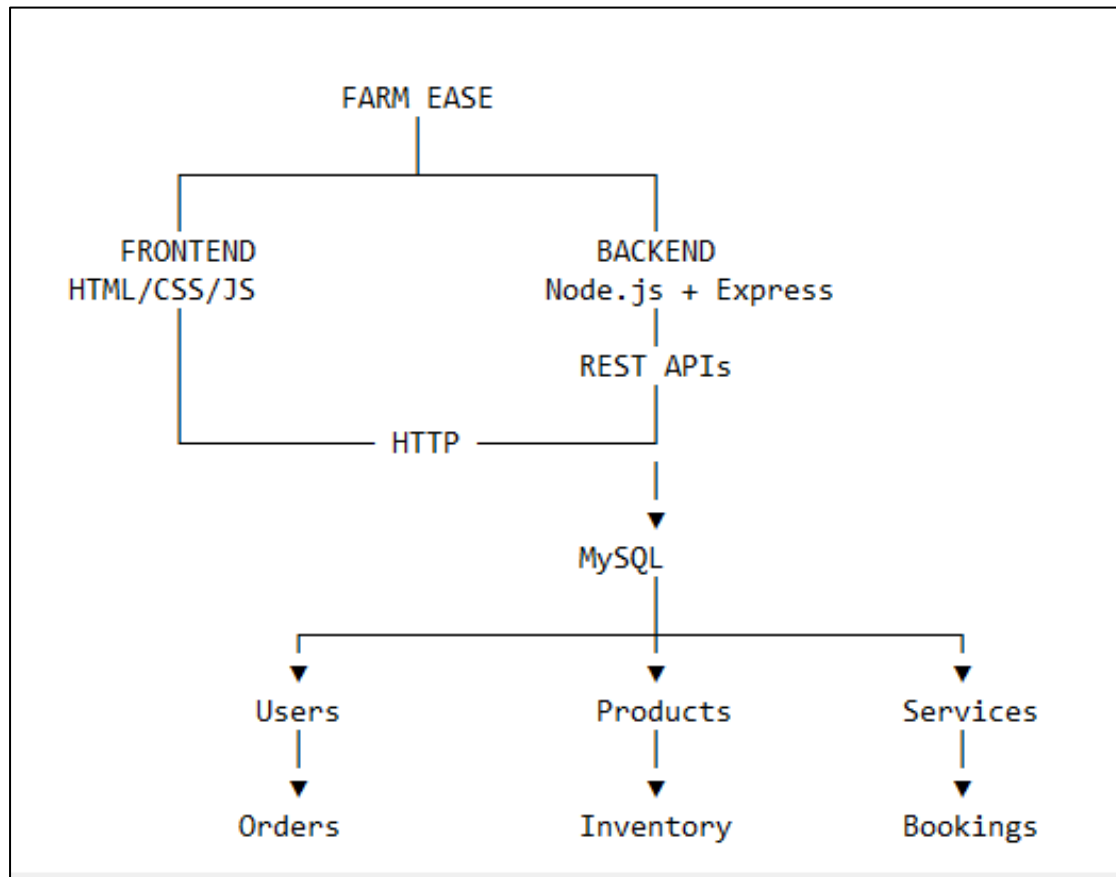
ER DIAGRAM



Relational Diagram:



Flow Diagram:



BACKEND

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const bcrypt = require("bcrypt");
const db = require("../db");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(cors());
app.use(express.json());

// Health Check
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", message: "FarmEase API is running" });
});

// Public Stats (for landing page)
app.get("/api/stats/public", async (req, res) => {
  try {
    const [farmers] = await db.promise().query("SELECT COUNT(*) as count FROM users WHERE roll = 'user'");
    const [products] = await db.promise().query("SELECT COUNT(*) as count FROM products");
    const [services] = await db.promise().query("SELECT COUNT(*) as count FROM services");
    res.json({
      success: true,
      stats: {
        farmers: farmers[0].count,
        products: products[0].count,
        services: services[0].count
      }
    });
  } catch (error) {
    console.error("Public stats error:", error);
    res.status(500).json({ success: false, message: "Failed to fetch stats." });
  }
});

// =====
// AUTHENTICATION
// =====

// Register
app.post("/api/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;
```

```

    if (!name || !email || !password) {
      return res.status(400).json({
        success: false,
        message: "All fields are required."
      });
    }

    const cleanName = name.trim();
    const cleanEmail = email.trim().toLowerCase();

    // Check if email already registered
    const [existingUsers] = await db.promise().query(
      "SELECT id FROM users WHERE email = ?",
      [cleanEmail]
    );

    if (existingUsers.length > 0) {
      return res.status(409).json({
        success: false,
        message: "An account with this email already exists."
      });
    }

    // Hash password
    const hashedPassword = await bcrypt.hash(password, 10);

    // Insert new user into MySQL (database column is `roll`)
    await db.promise().query(
      `INSERT INTO users (name, email, password, roll)
      VALUES (?, ?, ?, 'user')`,
      [cleanName, cleanEmail, hashedPassword]
    );

    res.status(201).json({
      success: true,
      message: "Account created successfully! You can now log in."
    });
  } catch (error) {
    console.error("Registration error:", error);
    res.status(500).json({
      success: false,
      message: "Server error. Please try again later."
    });
  }
});

// Login
app.post("/api/login", async (req, res) => {
  try {
    const { email, password, role } = req.body;

```

```

if (!email || !password) {
    return res.status(400).json({
        success: false,
        message: "Email and password are required."
    });
}

const cleanEmail = email.trim().toLowerCase();

// Query user by email and optional role
let sql = "SELECT * FROM users WHERE email = ?";
const params = [cleanEmail];

if (role) {
    sql += " AND roll = ?";
    params.push(role);
}

const [results] = await db.promise().query(sql, params);

if (results.length === 0) {
    return res.status(401).json({
        success: false,
        message: "Invalid email, password or selected role."
    });
}

const user = results[0];

// Verify password with bcrypt or fallback to plain comparison
let passwordMatch = false;
try {
    passwordMatch = await bcrypt.compare(password, user.password);
} catch (e) {
    passwordMatch = false;
}

// Backward compatibility fallback for plain text passwords if any
if (!passwordMatch && user.password === password) {
    passwordMatch = true;
}

if (!passwordMatch) {
    return res.status(401).json({
        success: false,
        message: "Invalid email, password or selected role."
    });
}

res.json({

```

```

        success: true,
        message: "Login successful",
        user: {
            id: user.id,
            name: user.name,
            email: user.email,
            role: user.roll,
            phone: user.phone,
            address: user.address
        }
    });
} catch (error) {
    console.error("Login error:", error);
    res.status(500).json({
        success: false,
        message: "Server error. Please try again later."
    });
}
});

// =====
// PRODUCTS
// =====

// Get all products (with optional ?category= filter)
app.get("/api/products", async (req, res) => {
    try {
        const { category } = req.query;
        let sql = "SELECT * FROM products";
        const params = [];

        if (category && category !== "all") {
            sql += " WHERE category = ?";
            params.push(category);
        }

        sql += " ORDER BY id ASC";

        const [products] = await db.promise().query(sql, params);
        res.json({ success: true, products });
    } catch (error) {
        console.error("Error fetching products:", error);
        res.status(500).json({ success: false, message: "Failed to fetch
products." });
    }
});

// Get single product
app.get("/api/products/:id", async (req, res) => {
    try {
        const [products] = await db.promise().query(

```

```

        "SELECT * FROM products WHERE id = ?",
        [req.params.id]
    );

    if (products.length === 0) {
        return res.status(404).json({ success: false, message: "Product
not found." });
    }

    res.json({ success: true, product: products[0] });
} catch (error) {
    console.error("Error fetching product:", error);
    res.status(500).json({ success: false, message: "Failed to fetch
product." });
}
});

// =====
// SERVICES
// =====

// Get all services
app.get("/api/services", async (req, res) => {
    try {
        const [services] = await db.promise().query("SELECT * FROM services
ORDER BY id ASC");
        res.json({ success: true, services });
    } catch (error) {
        console.error("Error fetching services:", error);
        res.status(500).json({ success: false, message: "Failed to fetch
services." });
    }
});

// Book a service
app.post("/api/services/book", async (req, res) => {
    try {
        const { user_id, service_id, booking_date } = req.body;

        if (!user_id || !service_id) {
            return res.status(400).json({
                success: false,
                message: "User ID and Service ID are required."
            });
        }

        const date = booking_date || new Date().toISOString().split("T")[0];

        const [result] = await db.promise().query(
            `INSERT INTO service_bookings (user_id, service_id, booking_date,
status)`

```

```

        VALUES (?, ?, ?, 'Booked')`,
        [user_id, service_id, date]
    );

    res.status(201).json({
        success: true,
        message: "Service booked successfully!",
        bookingId: result.insertId
    });
} catch (error) {
    console.error("Service booking error:", error);
    res.status(500).json({
        success: false,
        message: "Failed to book service."
    });
}
});

// Get user booked services
app.get("/api/services/user/:userId", async (req, res) => {
    try {
        const sql = `
            SELECT sb.id, sb.booking_date, sb.status, sb.created_at,
                   s.id as service_id, s.name, s.description, s.price
            FROM service_bookings sb
            JOIN services s ON sb.service_id = s.id
            WHERE sb.user_id = ?
            ORDER BY sb.id DESC
        `;
        const [services] = await db.promise().query(sql, [req.params.userId]);
        res.json({ success: true, services });
    } catch (error) {
        console.error("Error fetching user services:", error);
        res.status(500).json({ success: false, message: "Failed to fetch
booked services." });
    }
});

// =====
// ORDERS
// =====

// Place Order
app.post("/api/orders", async (req, res) => {
    const connection = await db.promise().getConnection();
    try {
        await connection.beginTransaction();

        const {
            user_id,
            product_id,

```

```

        quantity = 1,
        price,
        total_amount,
        payment_method = "Card",
        delivery_address = ""
    } = req.body;

    if (!user_id) {
        await connection.rollback();
        connection.release();
        return res.status(400).json({
            success: false,
            message: "User must be logged in to place an order."
        });
    }

    const finalTotal = total_amount || price || 0;

    // Insert order
    const [orderResult] = await connection.query(
        `INSERT INTO orders
        (user_id, total_amount, payment_method, payment_status,
order_status, delivery_address)
        VALUES (?, ?, ?, 'Paid', 'Confirmed', ?)` ,
        [user_id, finalTotal, payment_method, delivery_address]
    );

    const orderId = orderResult.insertId;

    // Insert order items
    // Support either single product or array of items
    if (req.body.items && Array.isArray(req.body.items)) {
        for (const item of req.body.items) {
            await connection.query(
                `INSERT INTO order_items (order_id, product_id, quantity,
price)
                VALUES (?, ?, ?, ?)` ,
                [orderId, item.product_id, item.quantity || 1, item.price]
            );
        }
    } else if (product_id) {
        await connection.query(
            `INSERT INTO order_items (order_id, product_id, quantity,
price)
            VALUES (?, ?, ?, ?)` ,
            [orderId, product_id, quantity, price || finalTotal]
        );
    }

    await connection.commit();
    connection.release();

```

```

        res.status(201).json({
            success: true,
            message: "Order placed successfully!",
            orderId: "FE" + orderId.toString().padStart(6, "0"),
            rawOrderId: orderId
        });
    } catch (error) {
        await connection.rollback();
        connection.release();
        console.error("Order error:", error);
        res.status(500).json({
            success: false,
            message: "Failed to place order."
        });
    }
});

// Get user orders
app.get("/api/orders/user/:userId", async (req, res) => {
    try {
        const sql = `
            SELECT o.id, o.total_amount, o.payment_method, o.payment_status,
            o.order_status,
                o.delivery_address, o.created_at,
                oi.product_id, oi.quantity, oi.price,
                p.name as product_name, p.category as product_category
            FROM orders o
            LEFT JOIN order_items oi ON o.id = oi.order_id
            LEFT JOIN products p ON oi.product_id = p.id
            WHERE o.user_id = ?
            ORDER BY o.id DESC
        `;

        const [rows] = await db.promise().query(sql, [req.params.userId]);

        // Group rows by order id
        const ordersMap = new Map();
        for (const row of rows) {
            if (!ordersMap.has(row.id)) {
                ordersMap.set(row.id, {
                    id: "FE" + row.id.toString().padStart(6, "0"),
                    rawId: row.id,
                    total_amount: Number(row.total_amount),
                    payment_method: row.payment_method,
                    payment_status: row.payment_status,
                    order_status: row.order_status,
                    delivery_address: row.delivery_address,
                    created_at: row.created_at,
                    product: row.product_name || "Farm Product",
                    price: Number(row.total_amount),

```

```

        status: row.order_status,
        date: new Date(row.created_at).toLocaleDateString(),
        items: []
    });
}

if (row.product_id) {
    ordersMap.get(row.id).items.push({
        product_id: row.product_id,
        name: row.product_name,
        quantity: row.quantity,
        price: Number(row.price)
    });
}
}

const orders = Array.from(ordersMap.values());
res.json({ success: true, orders });
} catch (error) {
    console.error("Error fetching user orders:", error);
    res.status(500).json({ success: false, message: "Failed to fetch
orders." });
}
});

// =====
// ADMIN PANEL
// =====

// Admin Stats
app.get("/api/admin/stats", async (req, res) => {
    try {
        const [farmers] = await db.promise().query(
            "SELECT COUNT(*) as totalFarmers FROM users WHERE roll = 'user'"
        );
        const [orders] = await db.promise().query(
            "SELECT COUNT(*) as totalOrders, COALESCE(SUM(total_amount), 0) as
totalRevenue FROM orders"
        );
        const [services] = await db.promise().query(
            "SELECT COUNT(*) as totalServices FROM service_bookings"
        );

        res.json({
            success: true,
            stats: {
                totalFarmers: farmers[0].totalFarmers,
                totalOrders: orders[0].totalOrders,
                totalRevenue: Number(orders[0].totalRevenue),
                totalServices: services[0].totalServices
            }
        });
    }
});

```

```

    });
  } catch (error) {
    console.error("Error fetching admin stats:", error);
    res.status(500).json({ success: false, message: "Failed to fetch admin
stats." });
  }
});

// Admin Orders
app.get("/api/admin/orders", async (req, res) => {
  try {
    const sql = `
      SELECT o.id, o.total_amount, o.payment_method, o.payment_status,
o.order_status,
      o.delivery_address, o.created_at,
      u.name as customer_name, u.email as customer_email, u.phone
as customer_phone,
      p.name as product_name, oi.quantity, oi.price as item_price
      FROM orders o
      JOIN users u ON o.user_id = u.id
      LEFT JOIN order_items oi ON o.id = oi.order_id
      LEFT JOIN products p ON oi.product_id = p.id
      ORDER BY o.id DESC
    `;

    const [rows] = await db.promise().query(sql);

    const ordersMap = new Map();
    for (const row of rows) {
      if (!ordersMap.has(row.id)) {
        ordersMap.set(row.id, {
          id: "FE" + row.id.toString().padStart(6, "0"),
          rawId: row.id,
          product: row.product_name || "Farm Product",
          customer: row.customer_name,
          customer_email: row.customer_email,
          customer_phone: row.customer_phone,
          delivery_address: row.delivery_address,
          price: Number(row.total_amount),
          payment_method: row.payment_method,
          status: row.order_status,
          date: new Date(row.created_at).toLocaleDateString()
        });
      }
    }

    res.json({ success: true, orders: Array.from(ordersMap.values()) });
  } catch (error) {
    console.error("Error fetching admin orders:", error);
    res.status(500).json({ success: false, message: "Failed to fetch admin
orders." });
  }
});

```

```

    }
  });

  // Admin Services
  app.get("/api/admin/services", async (req, res) => {
    try {
      const sql = `
        SELECT sb.id, sb.booking_date, sb.status, sb.created_at,
              u.name as customer_name, u.email as customer_email, u.phone
as customer_phone,
              s.name as service_name, s.price as service_price
        FROM service_bookings sb
        JOIN users u ON sb.user_id = u.id
        JOIN services s ON sb.service_id = s.id
        ORDER BY sb.id DESC
      `;

      const [rows] = await db.promise().query(sql);

      const formatted = rows.map(r => ({
        id: "SRV" + r.id.toString().padStart(6, "0"),
        name: r.service_name,
        customer: r.customer_name,
        customer_email: r.customer_email,
        customer_phone: r.customer_phone,
        price: Number(r.service_price),
        date: r.booking_date ? new
Date(r.booking_date).toLocaleDateString() : new
Date(r.created_at).toLocaleDateString(),
        status: r.status
      }));

      res.json({ success: true, services: formatted });
    } catch (error) {
      console.error("Error fetching admin services:", error);
      res.status(500).json({ success: false, message: "Failed to fetch admin
services." });
    }
  });

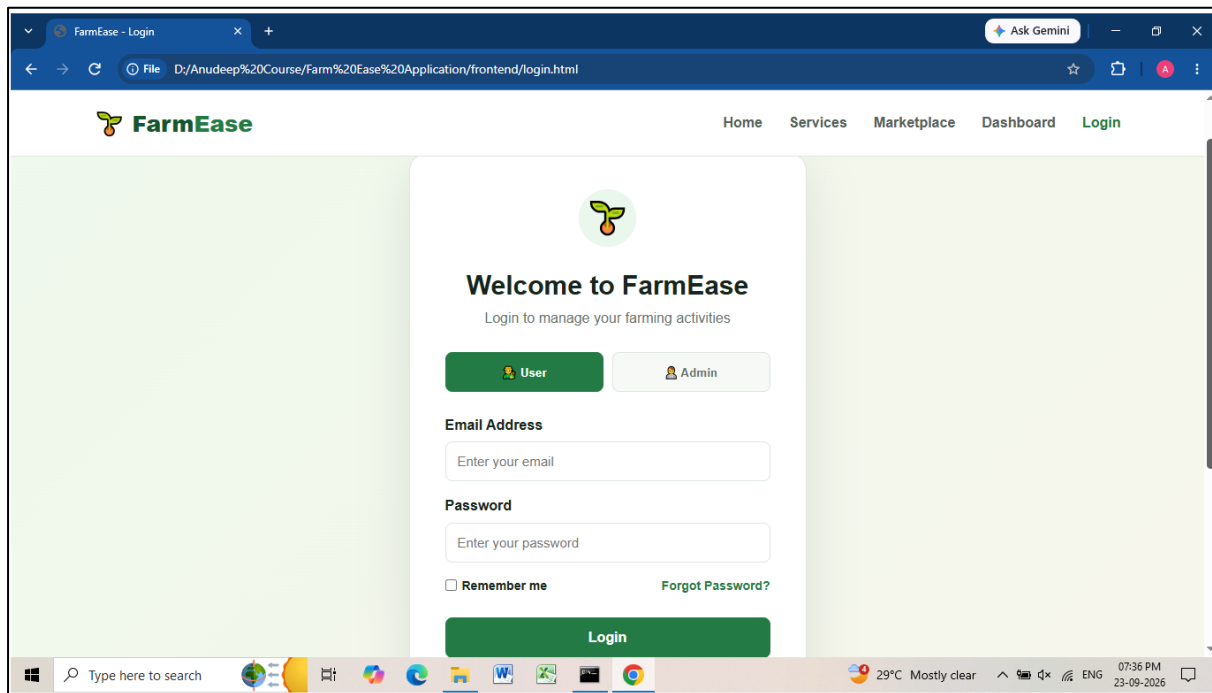
  // =====
  // START SERVER
  // =====

  app.listen(PORT, () => {
    console.log(`FarmEase server running at http://localhost:${PORT}`);
  });

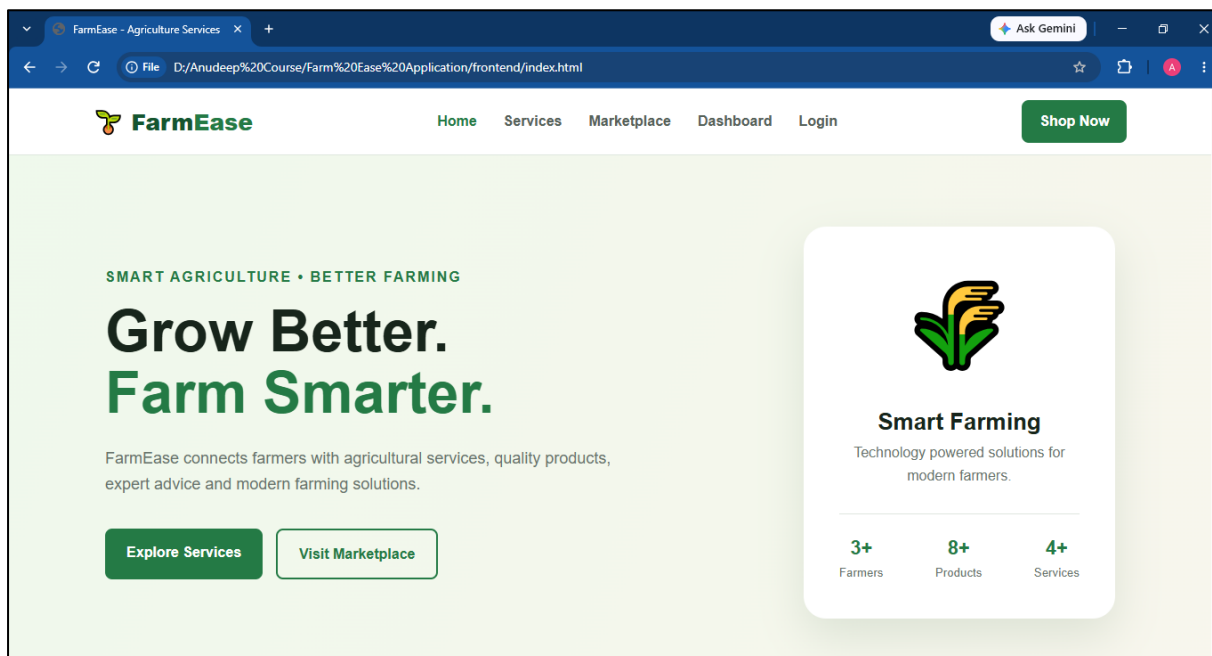
```

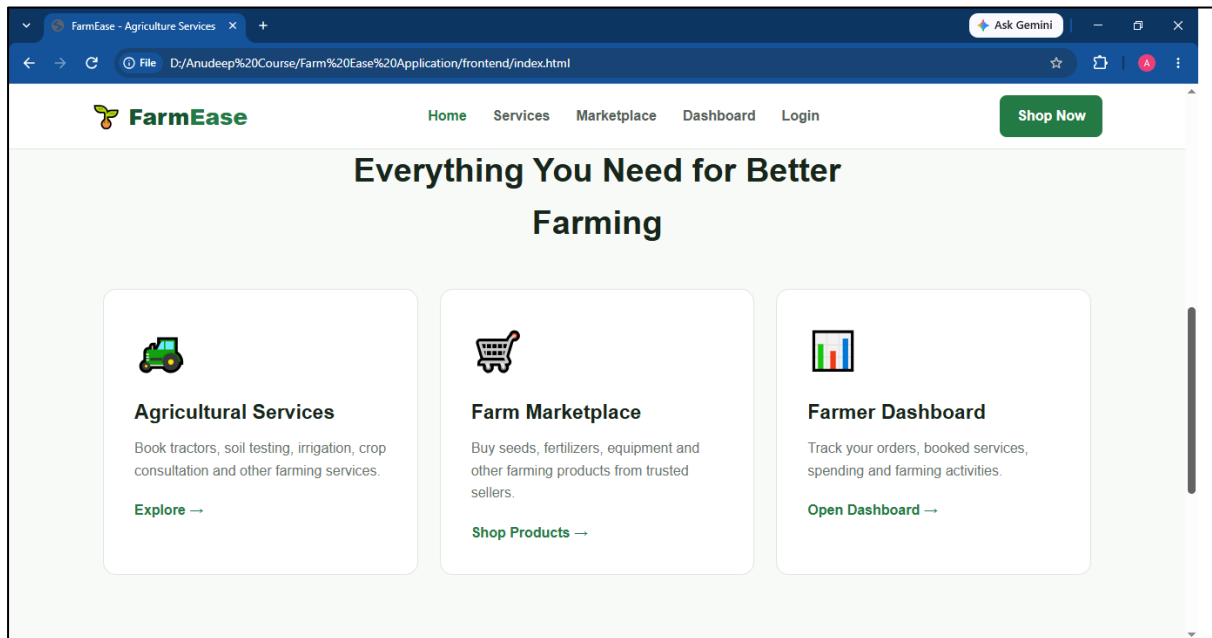
FRONTEND

1.Users/Admin Login Page:

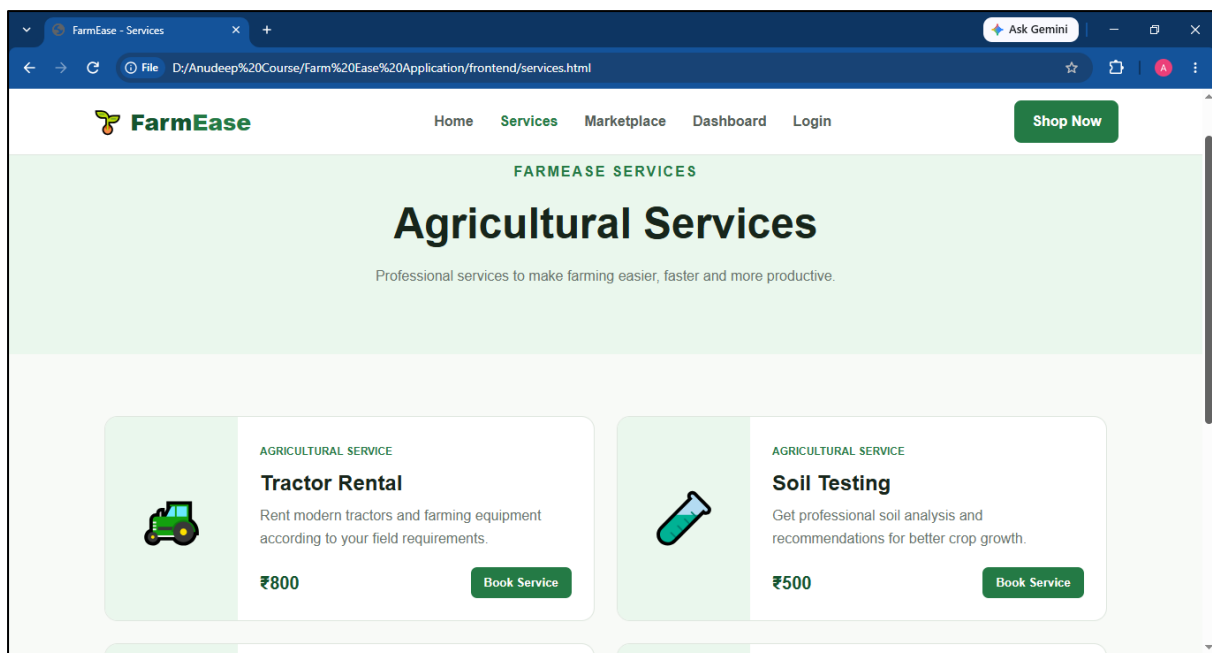


2.Home Page:

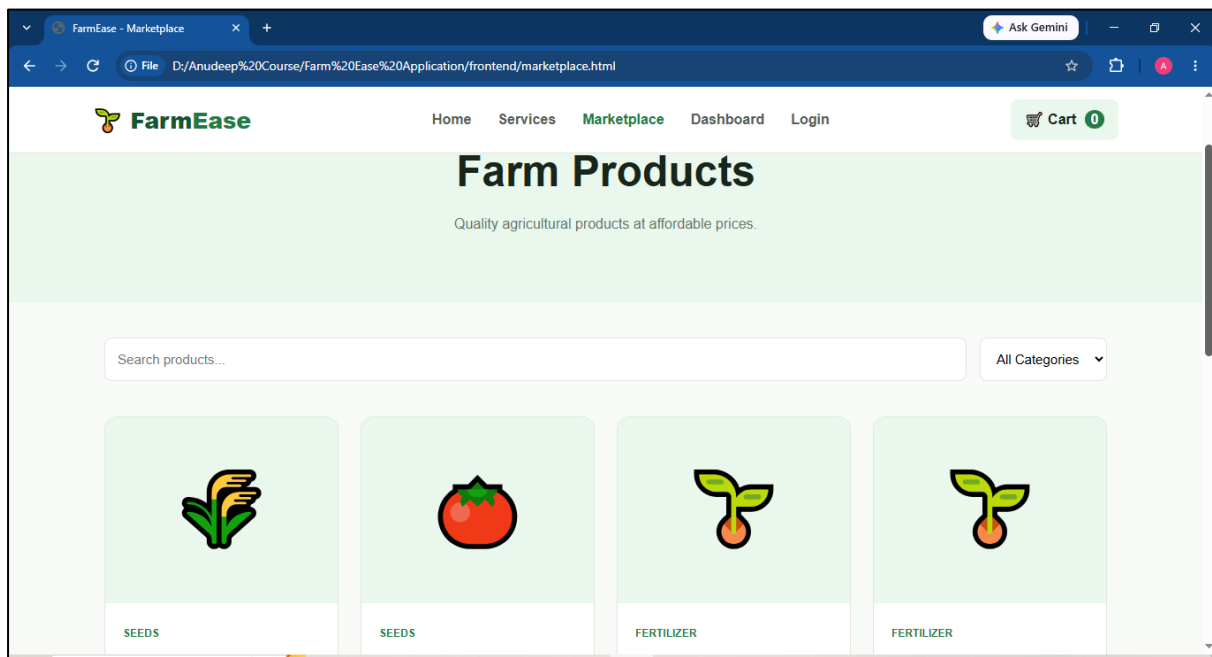




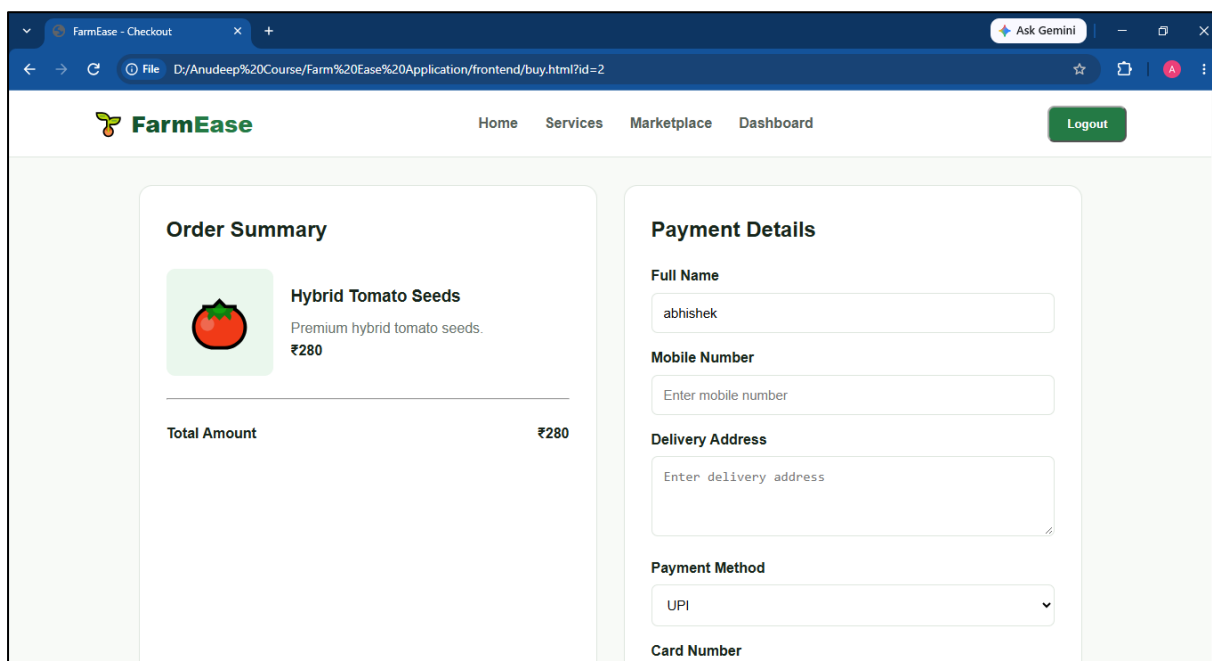
3.Services:



4. Marketplace:



5. Order summary:



6. User Dashboard:

The screenshot shows the FarmEase User Dashboard for user 'abhishek'. The dashboard includes a navigation bar with links to Home, Services, Marketplace, and Dashboard, along with a user profile icon and a Logout button. The main content area features a header for 'abhishek's Dashboard' with a sub-header 'Manage your FarmEase activities from one place.' Below this, there are three summary cards: 'Total Orders' (0), 'Services Booked' (2), and 'Total Spent' (₹3300). The 'Recent Orders' section shows 'No orders yet' with a 'Shop Products' button. The 'Booked Services' section lists 'Tractor Rental' (₹800.00, Booked) and 'Irrigation Setup' (₹2500.00, Booked).

Category	Value
Total Orders	0
Services Booked	2
Total Spent	₹3300

Service	Amount	Status
Tractor Rental	₹800.00	Booked
Irrigation Setup	₹2500.00	Booked

7. Admin Dashboard:

The screenshot shows the FarmEase Admin Dashboard. The dashboard includes a navigation bar with links to Home, Services, Marketplace, and Admin, along with a user profile icon and a Logout button. The main content area features a header for 'Admin Dashboard' with a sub-header 'Manage FarmEase operations.' Below this, there are three summary cards: 'Registered Farmers' (3), 'Total Orders' (2), and 'Revenue' (₹1010). The 'Recent Orders' section shows 'Hybrid Tomato Seeds' (₹560, Confirmed) with details like '#FE000002' and 'Customer: Suresh Patel'. The 'Services' section lists 'Tractor Rental' (₹800, Booked) with details like '22/9/2026' and 'Customer: abhishek'.

Category	Value
Registered Farmers	3
Total Orders	2
Revenue	₹1010

Order/Service	Amount	Status
Hybrid Tomato Seeds	₹560	Confirmed
Tractor Rental	₹800	Booked

APPLICATION FEATURES

- User registration with database storage.
- Password hashing using bcrypt.
- User login with password verification.
- Agricultural product marketplace.
- Dynamic product selection and product details.
- Quantity-based subtotal and total calculation.
- Delivery charge calculation.
- Delivery address collection.
- Agricultural service listing.
- Service booking functionality.
- MySQL relational database management.

User Registration API

The registration process uses an Express API endpoint such as POST /api/register. The frontend collects the user's name, email, password, and confirmation password. The request is sent to the backend using fetch

User Login

During login, the frontend sends the user's email and password to the backend. The backend retrieves the corresponding user record and uses bcrypt.compare to verify the password against the stored hash. A successful verification allows the application to continue with the authenticated user flow.

Product and Marketplace Flow

1. The user opens the FarmEase marketplace.
2. The application displays available agricultural products.
3. When the user selects Buy, the selected product identifier is passed to the buy page.
4. The buy page reads the product identifier from the URL and loads the corresponding product details.
5. The user selects the required quantity and enters delivery information.

Service Booking Flow

The services module allows users to view agriculture-related services and submit a booking. The booking is associated with both the user and the selected service so that the database can maintain a record of who requested which service.

FUTURE SCOPE

FarmEase can be extended with additional features to make the application more useful for farmers, customers, vendors, and agricultural service providers.

- Online payment gateway integration for UPI, cards, and other digital payment methods.
- Farmer and vendor dashboards for adding, editing, and managing products.
- Real-time order tracking with order status stages such as Pending, Confirmed, Shipped, and Delivered.
- Notifications through email, SMS, or push notifications for order and booking updates.
- Product reviews, ratings, and customer feedback.
- Search, filtering, sorting, and category-based marketplace browsing.
- Inventory and stock management for agricultural products.
- Advanced service-provider dashboards for managing bookings.
- Multilingual support for users from different regions.
- Mobile application support using a mobile-friendly framework.
- Analytics dashboards for sales, orders, products, and service bookings.
- Cloud deployment and scalable hosting for larger numbers of users.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of the FarmEase project. Their valuable guidance, suggestions, and encouragement helped us complete this project successfully.

We are thankful to our project guide and faculty members for providing us with the necessary knowledge, direction, and support during the different stages of the project. Their feedback helped us improve our understanding of web development, backend programming, database management, and software development practices.

We would also like to thank our department for providing the resources and learning environment required to carry out this project. We are grateful to our friends and classmates for their valuable suggestions, assistance, and encouragement.

Finally, we would like to thank our family members for their constant support, patience, and motivation throughout the development of the FarmEase application.

BIBLIOGRAPHY

- Mozilla Developer Network (MDN) Web Docs — References for HTML, CSS, JavaScript, and web development.
- Node.js Documentation — Reference material for server-side JavaScript development.
- Express.js Documentation — Documentation for creating web servers and REST APIs.
- MySQL Documentation — Reference material for relational database design, SQL queries, and database management.
- mysql2 Documentation — Reference material for connecting Node.js applications with MySQL.
- bcrypt Documentation — Reference material for password hashing and password verification.
- Git Documentation — Reference material for version control.
- GitHub Documentation — Reference material for repository management and project collaboration.
- W3Schools — Online tutorials and references for HTML, CSS, JavaScript, and SQL.
- OpenAI ChatGPT — Used as an AI-assisted learning and development resource for understanding concepts, troubleshooting code, debugging errors, and improving the implementation of the project.